

Yesterday continue

When we run

when we click Books

← → ↺ 📑 localhost:44343 🔍 ☆ 🧑 Verify it's you ⋮

LibraryManagement Books Add Book

Books List

Title	Author	ISBN	Published Date	Availability	Actions
The Pragmatic Programmer	Andrew Hunt and David Thomas	978-0201616224	2021-10-30	Available	Details Edit Delete Borrow
Design Pattern using C#	Robert C. Martin	978-0132350884	2023-08-01	Available	Details Edit Delete Borrow
Mastering ASP.NET Core	Pranaya Kumar Rout	978-0451616235	2022-11-22	Available	Details Edit Delete Borrow
SQL Server with DBA	Rakesh Kumat	978-4562350123	2020-08-15	Available	Details Edit Delete Borrow

Add New Book

when we click add

← → ↺ 📑 localhost:44343/books/create 🔍 ✕ 🧑 Verify it's you ⋮

LibraryManagement Books Add Book

Add New Book

Title

Enter book title

Author

Enter author's name

ISBN

e.g., 978-1234567890

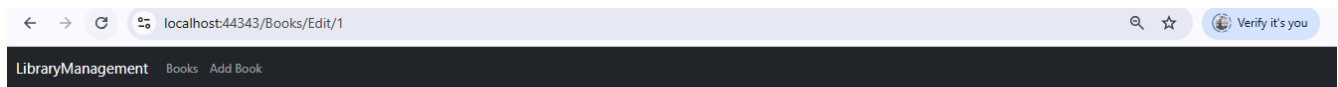
Published Date

mm/dd/yyyy

Add Book

Cancel

when we click edit



 Edit Book

Title

The Pragmatic Programmer

Author

Andrew Hunt and David Thomas

ISBN

978-0201616224

Published Date

10/30/2021

 Save Changes  Cancel

when we click delete

⚠️ Confirm Delete

Are you sure you want to delete this book?

Once deleted, this action cannot be undone.

Title:

The Pragmatic Programmer

Author:

Andrew Hunt and David Thomas

ISBN:

978-0201616224

Published Date:

2021-10-30

Availability:

Available

🗑️ Delete

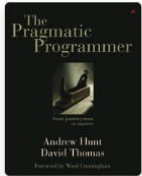
⌛ Cancel

when we click details

LibraryManagement

Books

Add Book



The Pragmatic Programmer

Author:

ISBN:

Published Date:

Availability:

Andrew Hunt and David Thomas

978-0201616224

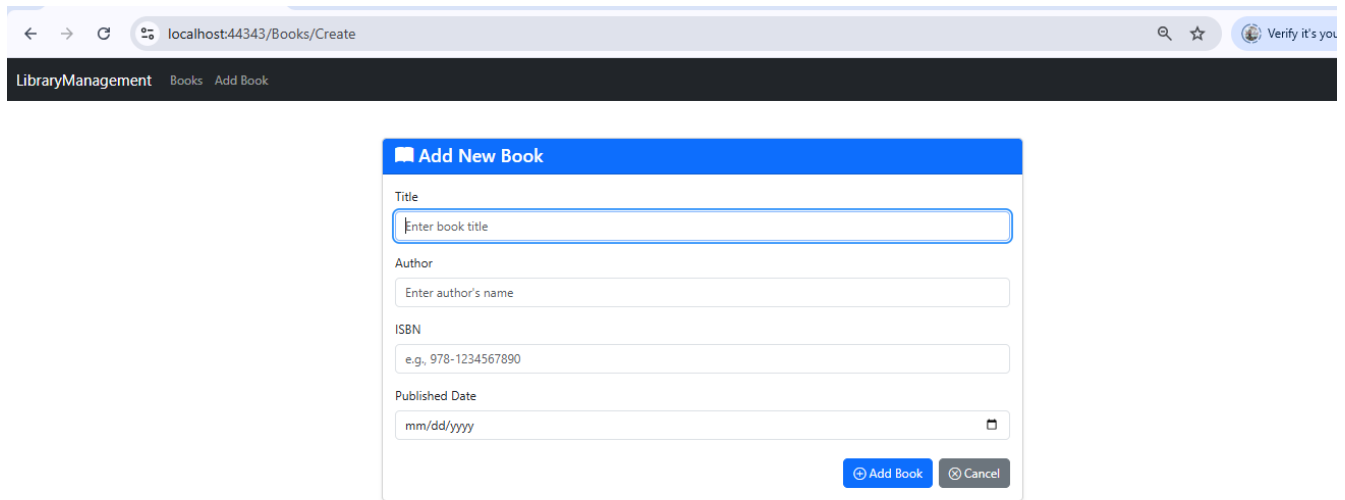
2021-10-30

Available

 Edit

 Back to List

when we click add new books then



The screenshot shows a web browser at the URL `localhost:44343/Books/Create`. The browser's address bar includes navigation icons and a search icon. The page has a dark header with the text "LibraryManagement" and a breadcrumb trail "Books > Add Book". The main content area features a white modal form titled "Add New Book" with a blue header bar. The form contains four input fields: "Title" with a placeholder "Enter book title", "Author" with a placeholder "Enter author's name", "ISBN" with a placeholder "e.g., 978-1234567890", and "Published Date" with a placeholder "mm/dd/yyyy" and a calendar icon. At the bottom right of the form are two buttons: a blue "Add Book" button and a grey "Cancel" button.

all validation and cancel button working properly

START FROM HERE

Next session tomorrow 21/07/26

6) Borrow Book Page

This page allows the admin to process a borrowing request for a specific book. It displays the book's details and requires information about the borrower (such as their name and contact information). If the book is available, it can be marked as borrowed, and a borrow record will be created.

Borrow Book

The Pragmatic Programmer

BorrowerName

Pranaya

BorrowerEmail

pranayakumar777@Example.com

Phone

1234567890

Confirm Borrow

Cancel

7)Return Book Page

This page facilitates the return process for a borrowed book. It displays details of the borrow record, such as the borrower's name and the borrow date, and allows the administrator to mark the book as returned

Return Book

The Pragmatic Programmer

Borrower Name: Pranaya

Borrow Date: 2024-10-02 07:09:18

Confirm Return

Cancel

8) Book Already Returned Page

This page is shown when an admin attempts to process a return for a book already returned. It notifies the admin that the return action is unnecessary as the borrowing record indicates that the book has already been returned.

Book Already Returned

The borrow record for 'Design Pattern using C#' has already been returned.

[Back to Book List](#)

Creating View Models for Managing Book Borrow and Return:

So, first, create a folder named **ViewModels** in the Project root directory, and inside this folder, we will create all our View Models.

BorrowViewModel

Create a class file named **BorrowViewModel.cs** within the **ViewModels** folder, and then copy and paste the following code. This serves as a view model used for borrowing books. It encapsulates the data required when a user borrows a book, including BookId, BookTitle, BorrowerName, BorrowerEmail, and Phone. It ensures that necessary information is collected from the user when borrowing a book.

```
using Microsoft.AspNetCore.Mvc.ModelBinding;
using System.ComponentModel.DataAnnotations;

namespace LMSysstem.ViewModels
{
    public class BorrowViewModel
    {
        [Required]
        public int BookId { get; set; }
        [BindNever]
        public string? BookTitle { get; set; }
        [Required(ErrorMessage = "Your name is required.")]
        [StringLength(100, ErrorMessage = "Name cannot exceed 100 characters.")]
        public string ?BorrowerName { get; set; }
        [Required(ErrorMessage = "Your email is required.")]
        [EmailAddress(ErrorMessage = "Invalid email address.")]
        public string ?BorrowerEmail { get; set; }
        [Required(ErrorMessage = "Your Phone Number is required.")]
        [Phone(ErrorMessage = "Invalid Phone Number")]
        public string ?Phone { get; set; }
    }
}
```

ReturnViewModel

Create a class file named **ReturnViewModel.cs** within the ViewModels folder, and then copy and paste the following code. This class serves as a view model for returning borrowed books. It includes properties like BorrowRecordId, BookId, BookTitle, BorrowerName, and BorrowDate. It helps display the necessary information during the return process.

```
using Microsoft.AspNetCore.Mvc.ModelBinding;
using System.ComponentModel.DataAnnotations;

namespace LMSysSystem.ViewModels
{
    public class ReturnViewModel
    {
        [Required]
        public int BorrowRecordId { get; set; }
        [BindNever]
        public string? BookTitle { get; set; }
        [BindNever]
        public string? BorrowerName { get; set; }
        [BindNever]
        public DateTime? BorrowDate { get; set; }
    }
}
```

Creating the BorrowController

Create an empty MVC controller named **BorrowController** within the Controllers folder, then copy and paste the following code. This controller manages book borrowing and return. It handles actions such as displaying the borrow form (Create GET), processing the borrowing action (Create POST), displaying the return confirmation (Return GET), and processing the return action (Return POST). It updates the books' IsAvailable status and manages BorrowRecord entries accordingly.

```
using LMSysSystem.Models;
using LMSysSystem.ViewModels;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace LMSysSystem.Controllers
{
    public class BorrowController : Controller
    {
        private readonly LibraryContext _context;
        public BorrowController(LibraryContext context)
        {
            _context = context;
        }
        // Displays the borrow form for a specific book.
        // GET: Borrow/Create/5
    }
}
```



```

public async Task<IActionResult> Create(int? bookId)
{
    if (bookId == null || bookId == 0)
    {
        TempData["ErrorMessage"] = "Book ID was not provided for borrowing.";
        return View("NotFound");
    }
    try
    {
        var book = await _context.Books13.FindAsync(bookId);
        if (book == null)
        {
            TempData["ErrorMessage"] = $"No book found with ID {bookId} to borrow.";
            return View("NotFound");
        }
        if (!book.IsAvailable)
        {
            TempData["ErrorMessage"] = $"The book '{book.Title}' is currently not
available for borrowing.";
            return View("NotAvailable");
        }
        var borrowViewModel = new BorrowViewModel
        {
            BookId = book.BookId,
            BookTitle = book.Title
        };
        return View(borrowViewModel);
    }
    catch (Exception ex)
    {
        TempData["ErrorMessage"] = "An error occurred while loading the borrow form.";
        return View("Error");
    }
}

// Processes the borrowing action, creates a BorrowRecord, updates the book's
availability
// POST: Borrow/Create
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(BorrowViewModel model)
{
    if (!ModelState.IsValid)
    {
        return View(model);
    }
}

```

```

    }
    try
    {
        var book = await _context.Books13.FindAsync(model.BookId);
        if (book == null)
        {
            TempData["ErrorMessage"] = $"No book found with ID {model.BookId} to borrow.";
            return View("NotFound");
        }
        if (!book.IsAvailable)
        {
            TempData["ErrorMessage"] = $"The book '{book.Title}' is already borrowed.";
            return View("NotAvailable");
        }
        var borrowRecord = new BorrowRecord
        {
            BookId = book.BookId,
            BorrowerName = model.BorrowerName,
            BorrowerEmail = model.BorrowerEmail,
            Phone = model.Phone,
            BorrowDate = DateTime.UtcNow
        };
        // Update the book's availability
        book.IsAvailable = false;
        _context.BorrowRecords13.Add(borrowRecord);
        await _context.SaveChangesAsync();
        TempData["SuccessMessage"] = $"Successfully borrowed the book: {book.Title}.";
        return RedirectToAction("Index", "Books");
    }
    catch (Exception ex)
    {
        TempData["ErrorMessage"] = "An error occurred while processing the borrowing
action.";
        return View("Error");
    }
}

// Displays the return confirmation for a specific borrow record
// GET: Borrow/Return/5
public async Task<IActionResult> Return(int? borrowRecordId)
{
    if (borrowRecordId == null || borrowRecordId == 0)
    {
        TempData["ErrorMessage"] = "Borrow Record ID was not provided for returning.";
    }
}

```

```

        return View("NotFound");
    }
    try
    {
        var borrowRecord = await _context.BorrowRecords13
            .Include(br => br.Book)
            .FirstOrDefaultAsync(br => br.BorrowRecordId == borrowRecordId);
        if (borrowRecord == null)
        {
            TempData["ErrorMessage"] = $"No borrow record found with ID {borrowRecordId}
to return.";
            return View("NotFound");
        }
        if (borrowRecord.ReturnDate != null)
        {
            TempData["ErrorMessage"] = $"The borrow record for '{borrowRecord.Book.Title}'
has already been returned.";
            return View("AlreadyReturned");
        }
        var returnViewModel = new ReturnViewModel
        {
            BorrowRecordId = borrowRecord.BorrowRecordId,
            BookTitle = borrowRecord.Book.Title,
            BorrowerName = borrowRecord.BorrowerName,
            BorrowDate = borrowRecord.BorrowDate
        };
        return View(returnViewModel);
    }
    catch (Exception ex)
    {
        TempData["ErrorMessage"] = "An error occurred while loading the return
confirmation.";
        return View("Error");
    }
}

// Processes the return action, updates the BorrowRecord with the return date, updates
the book's availability
// POST: Borrow/Return/5
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Return(ReturnViewModel model)
{
    if (!ModelState.IsValid)
    {
        return View(model);
    }
}

```

```

    }
    try
    {
        var borrowRecord = await _context.BorrowRecords13
            .Include(br => br.Book)
            .FirstOrDefaultAsync(br => br.BorrowRecordId == model.BorrowRecordId);
        if (borrowRecord == null)
        {
            TempData["ErrorMessage"] = $"No borrow record found with ID
{model.BorrowRecordId} to return.";
            return View("NotFound");
        }
        if (borrowRecord.ReturnDate != null)
        {
            TempData["ErrorMessage"] = $"The borrow record for '{borrowRecord.Book.Title}'
has already been returned.";
            return View("AlreadyReturned");
        }
        // Update the borrow record
        borrowRecord.ReturnDate = DateTime.UtcNow;
        // Update the book's availability
        borrowRecord.Book.IsAvailable = true;
        await _context.SaveChangesAsync();
        TempData["SuccessMessage"] = $"Successfully returned the book:
{borrowRecord.Book.Title}.";
        return RedirectToAction("Index", "Books");
    }
    catch (Exception ex)
    {
        TempData["ErrorMessage"] = "An error occurred while processing the return
action.";
        return View("Error");
    }
}
}
}

```

Designing the Views for Borrowing and Returning Books

Now, we will create views for borrowing and returning books and custom error views to handle different error scenarios.

Borrow View

Create a view file named **Create.cshtml** within the **Views/Borrow** folder, and then copy and paste the following code. The following view provides a form for borrowing a specific book. It includes fields for the borrower's name, email, and phone number and displays the title of the book being borrowed.

```
@model LMSysstem.ViewModels.BorrowViewModel
@{
    ViewData["Title"] = "Borrow Book";
}

<div class="container mt-5">
    <h2>Borrow Book</h2>
    <div class="card">
        <div class="card-header">
            <strong>@Model.BookTitle</strong>
        </div>
        <div class="card-body">
            <form asp-action="Create" method="post">
                <input type="hidden" asp-for="BookId" />
                @* <input type="hidden" asp-for="BookTitle" /> *@

                <div class="mb-3">
                    <label asp-for="BorrowerName" class="form-label"></label>
                    <input asp-for="BorrowerName" class="form-control" />
                    <span asp-validation-for="BorrowerName" class="text-danger"></span>
                </div>

                <div class="mb-3">
                    <label asp-for="BorrowerEmail" class="form-label"></label>
                    <input asp-for="BorrowerEmail" class="form-control" />
                    <span asp-validation-for="BorrowerEmail" class="text-danger"></span>
                </div>

                <div class="mb-3">
                    <label asp-for="Phone" class="form-label"></label>
                    <input asp-for="Phone" class="form-control" />
                    <span asp-validation-for="Phone" class="text-danger"></span>
                </div>

                <button type="submit" class="btn btn-primary">Confirm Borrow</button>
                <a asp-action="Index" asp-controller="Books" class="btn btn-
secondary">Cancel</a>
            </form>
        </div>
    </div>
</div>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Return View

Create a view file named **Return.cshtml** within the **Views/Borrow** folder, and then copy and paste the following code. It presents a confirmation page for returning a borrowed book. It displays details about the borrow record and allows the user to confirm the return action.

```
@model LMSysTem.ViewModels.ReturnViewModel
```

```
@{
```

```
    ViewData["Title"] = "Return Book";
```

```
}
```

```
<div class="container mt-5">
```

```
    <h2>Return Book</h2>
```

```
    <div class="card">
```

```
        <div class="card-header">
```

```
            <strong>@Model.BookTitle</strong>
```

```
        </div>
```

```
        <div class="card-body">
```

```
            <p><strong>Borrower Name:</strong> @Model.BorrowerName</p>
```

```
            <p><strong>Borrow Date:</strong> @Model.BorrowDate?.ToString("yyyy-MM-dd  
HH:mm:ss")</p>
```

```
        <form asp-action="Return" method="post">
```

```
            <input type="hidden" asp-for="BorrowRecordId" />
```

```
            <button type="submit" class="btn btn-success">Confirm Return</button>
```

```
            <a asp-action="Index" asp-controller="Books" class="btn btn-  
secondary">Cancel</a>
```

```
        </form>
```

```
    </div>
```

```
</div>
```

```
</div>
```

Not Available View

Create a view file named **NotAvailable.cshtml** within the **Views/Shared** folder, then copy and paste the following code. This view displays an error message when a user attempts to borrow a book that is not available. It informs the user that the book is currently unavailable and provides a link back to the book list.

```
@{
    ViewData["Title"] = "Book Not Available";
    var errorMessage = ViewBag.ErrorMessage as string;
}

<div class="container text-center mt-5">
    <h1 class="display-4 text-warning">Book Not Available</h1>
    <p class="lead">
        @if (!string.IsNullOrEmpty(errorMessage))
        {
            @errorMessage
        }
        else
        {
            <text>The book you are trying to borrow is currently not available.</text>
        }
    </p>
    <a asp-action="Index" asp-controller="Books" class="btn btn-primary">Back to Book
List</a>
</div>
}
```

Already Returned View

Create a view file named **AlreadyReturned.cshtml** within the **Views/Shared** folder, then copy and paste the following code. This view shows an error message if a user tries to return a book that has already been returned. It notifies the user of the issue and offers navigation back to the main list.

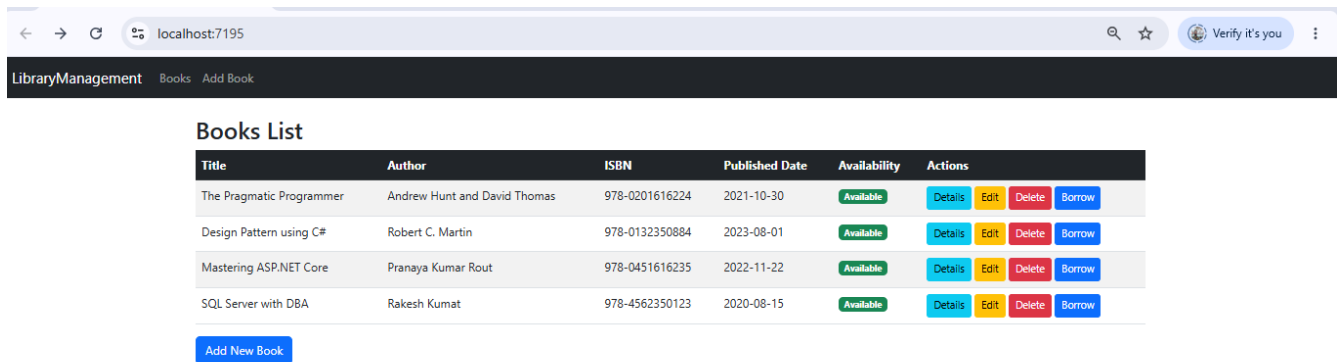
```
@{
    ViewData["Title"] = "Book Already Returned";
    var errorMessage = ViewBag.ErrorMessage as string;
}

<div class="container text-center mt-5">
    <h1 class="display-4 text-info">Book Already Returned</h1>
    <p class="lead">
        @if (!string.IsNullOrEmpty(errorMessage))
        {
            @errorMessage
        }
        else
    </p>

```

```
{
  <text>The borrow record for this book has already been returned.</text>
}
</p>
<a asp-action="Index" asp-controller="Books" class="btn btn-primary">Back to Book
List</a>
</div>
```

now when we run the application will get



LibraryManagement Books Add Book

Books List

Title	Author	ISBN	Published Date	Availability	Actions
The Pragmatic Programmer	Andrew Hunt and David Thomas	978-0201616224	2021-10-30	Available	Details Edit Delete Borrow
Design Pattern using C#	Robert C. Martin	978-0132350884	2023-08-01	Available	Details Edit Delete Borrow
Mastering ASP.NET Core	Pranaya Kumar Rout	978-0451616235	2022-11-22	Available	Details Edit Delete Borrow
SQL Server with DBA	Rakesh Kumat	978-4562350123	2020-08-15	Available	Details Edit Delete Borrow

[Add New Book](#)

when I will click first book for borrow button

like that when I click remaining borrow button click I will get this also

LibraryManagementBooksAdd Book

localhost:7195/Borrow/Create?bookId=1

Verify it's you

Borrow Book

The Pragmatic Programmer

BorrowerName

BorrowerEmail

Phone

Confirm Borrow

Cancel

now suppose I did 1 entry

LibraryManagementBooksAdd Book

Borrow Book

The Pragmatic Programmer

BorrowerName

rupesh

BorrowerEmail

phalnerupesh89@gmail.com

Phone

7718955817

Confirm Borrow

Cancel

and I will click confirm borrow

then I will get

← → ↻ 📑 localhost:7195 🔍 ☆ 🧑 Verify it's you ⋮

LibraryManagement Books Add Book

Successfully borrowed the book: The Pragmatic Programmer. ✕

Books List

Title	Author	ISBN	Published Date	Availability	Actions
The Pragmatic Programmer	Andrew Hunt and David Thomas	978-0201616224	2021-10-30	Borrowed	Details Edit Delete Return
Design Pattern using C#	Robert C. Martin	978-0132350884	2023-08-01	Available	Details Edit Delete Borrow
Mastering ASP.NET Core	Pranaya Kumar Rout	978-0451616235	2022-11-22	Available	Details Edit Delete Borrow
SQL Server with DBA	Rakesh Kumat	978-4562350123	2020-08-15	Available	Details Edit Delete Borrow

Add New Book

simultaneously check record for primary key and foreign key references

```
select * from Books13
select * from BorrowRecords13
```

like that I will do for 1 more borrow

← → ↻ 📑 localhost:7195/Borrow/Create?bookId=2 🔍 ☆ 🧑 Verify it's you ⋮

LibraryManagement Books Add Book

Borrow Book

Design Pattern using C#

BorrowerName

ramesh patil

BorrowerEmail

phalnerupesh89@gmail.com

Phone

7718955817

Confirm Borrow

Cancel

and I will click confirm borrow

then I will get

← → ↻ 📄 localhost:7195 🔍 ☆ 🧑 Verify it's you ⋮

LibraryManagement Books Add Book

Successfully borrowed the book: Design Pattern using C#. ✕

Books List

Title	Author	ISBN	Published Date	Availability	Actions
The Pragmatic Programmer	Andrew Hunt and David Thomas	978-0201616224	2021-10-30	Borrowed	Details Edit Delete Return
Design Pattern using C#	Robert C. Martin	978-0132350884	2023-08-01	Borrowed	Details Edit Delete Return
Mastering ASP.NET Core	Pranaya Kumar Rout	978-0451616235	2022-11-22	Available	Details Edit Delete Borrow
SQL Server with DBA	Rakesh Kumar	978-4562350123	2020-08-15	Available	Details Edit Delete Borrow

[Add New Book](#)

now suppose I have added 1 new entry

← → ↻ 📄 localhost:7195/books/create 🔍 ☆ 🧑 Verify it's you ⋮

LibraryManagement Books Add Book

Add New Book

Title

Author

ISBN

Published Date

[Add Book](#) [Cancel](#)

after clicking add book
I will get

← → ↻ 📄 localhost:7195 🔍 ☆ 🧑 Verify it's you ⋮

LibraryManagement Books Add Book

Successfully added the book: html. ✕

Books List

Title	Author	ISBN	Published Date	Availability	Actions
The Pragmatic Programmer	Andrew Hunt and David Thomas	978-0201616224	2021-10-30	Borrowed	Details Edit Delete Return
Design Pattern using C#	Robert C. Martin	978-0132350884	2023-08-01	Borrowed	Details Edit Delete Return
Mastering ASP.NET Core	Pranaya Kumar Rout	978-0451616235	2022-11-22	Available	Details Edit Delete Borrow
SQL Server with DBA	Rakesh Kumar	978-4562350123	2020-08-15	Available	Details Edit Delete Borrow
html	rajat	978-1234567895	2026-07-14	Available	Details Edit Delete Borrow

[Add New Book](#)

simultaneously check record for primary key and foreign key references

```
select * from Books13
select * from BorrowRecords13
```

important note for borrowed book check details

important note for available book check details

scenario for Book Not Available

scenario for Book already return

finally we have this

WE HAVE TWO MODULE IN PROJECT
BOOK AND BORROWER

